

QuizPod

An Intelligent Quiz & Learning Platform

Web Technology — Progress-1 Documentation

Name	Roll No.
Ayush Chauhan	2023UIT3134
Dev Aggarwal	2023UIT3136
Bhavya Agrawal	2023UIT3138
Rohan Singh	2023UIT3139
Payal	2023UIT3142
Mudit	2023UIT3149

Team Size: 6 Members

March 2026

Subject: Web Technology

Evaluation: Progress-1

GitHub: [DevAggarwal03/WebTech_Project](https://github.com/DevAggarwal03/WebTech_Project)

Contents

1	Topic of the Project	3
1.1	Problem Statement and Theme	3
1.2	Research on Existing Platforms	3
1.2.1	Universal Gaps Across All Platforms	4
1.2.2	Research on Unauthorised Access Prevention	4
2	Structure of the Project with Technology Stack	5
2.1	Project Structure — Pages and Sitemap	5
2.2	Ideation and Brainstorming Process	7
2.3	Technology Stack and Justification	9
2.4	Deployment Plan	9
2.5	Data Flow Diagrams (DFD)	9
2.5.1	Level 0 — Context Diagram	10
2.5.2	Level 1 — Major Processes	10
3	Implementation	10
3.1	Current Implementation Status ($\approx 50\%$)	10

3.2	Key Implementation Highlights	11
3.2.1	Quiz Arena — Full Interactive Quiz-Taking Experience	11
3.2.2	Dual-Role Authentication	11
3.2.3	Dashboard Personalisation	11
3.2.4	Component Architecture	12
4	Future Work and Teamwork	12
4.1	Remaining Work and Next Steps	12
4.2	Integration Components	13
4.3	Team Contribution	14
5	Additional Information	14
5.1	Key Differentiators from Existing Platforms	14
5.2	Project Repository	14
5.3	References	14

1. Topic of the Project

1.1 Problem Statement and Theme

Existing online quiz and learning platforms fail to genuinely optimise for *learning outcomes*. They prioritise engagement metrics (leaderboards, speed-based scoring) or administrative convenience, while students receive shallow feedback, no adaptive learning, and teachers get minimal actionable analytics.

Core Problem Areas:

- **Shallow feedback** — Platforms show scores but never explain *why* an answer was wrong or *what to study next*.
- **No adaptive difficulty** — Every student receives the same quiz regardless of skill level.
- **Academic integrity gaps** — Tab-switching, answer-sharing, and unauthorised quiz access are poorly addressed.
- **Unauthorised access** — Students not enrolled in a specific class can freely join if they obtain the access code, compromising quiz integrity.
- **Paywall exclusion** — Analytics, student tracking, and rich question types are locked behind expensive paid tiers.
- **No collaborative learning** — Quizzes are always competitive; team-based learning modes are absent.

Our Solution — QuizPod:

A web-based quiz platform that places learning outcomes at the centre. It provides rich feedback, teacher-controlled classrooms with unauthorised-student detection, detailed performance analytics, and lightweight engagement elements like XP and badges — all for free.

“The quiz platform that actually helps students learn, not just score — and actually helps teachers teach, not just grade.”

1.2 Research on Existing Platforms

We conducted detailed research on six major quiz/assessment platforms to identify gaps and opportunities.

Table 1: Analysis of Existing Quiz Platforms

Platform	Primary Strength	Core Audience	Key Shortcoming
Kahoot	Gamified live quizzes	K-12, higher ed	Speed > accuracy; free tier capped at 10 players
Quizlet	Flashcards & study modes	Students	Key modes behind paywall; ad-heavy free tier
Google Forms	Simple form-based quizzes	General	No native timer; easy misconfiguration exposes answers
Mentimeter	Real-time polling	Higher ed	Only 2 free interactive slides; weak analytics
Socrative	Formative live assessments	K-12 teachers	Capped at 50 students; MCQ-only
Typeform	Conversational form UX	Business/HR	Expensive per-response pricing; no auto-close

1.2.1 Universal Gaps Across All Platforms

1. No Adaptive Difficulty — questions never adjust based on student performance.
2. Shallow Feedback — most platforms show a score with no explanation.
3. No Spaced Repetition — no intelligent review scheduling.
4. Weak Academic Integrity — no lightweight anti-cheat mechanisms.
5. Paywalls block core features — analytics, tracking, and rich types are premium-only.
6. No Collaborative Quizzing — students always compete, never cooperate.
7. Poor Teacher Analytics — class averages but not per-concept diagnosis.
8. Rich Content Limitations — code, $\text{\LaTeX}$, diagrams not natively supported.
9. Fragmented Workflow — quiz creation, sharing, and grading are spread across tools.
10. No Offline Mode — all platforms require stable internet.

1.2.2 Research on Unauthorised Access Prevention

Table 2: Access Control Mechanisms Across Platforms

Platform	Access Mechanism	Gaps
Google Classroom	6–8 char class code; domain restriction	Code can be shared/leaked; no approval step
Kahoot	Temporary game PIN per session	No persistent “class” concept
Quizizz	Join code via joinmyquiz.com	Code-based, no approval
Moodle	Enrollment key, manual enrollment	Powerful but complex for teachers
Canvas LMS	SIS integration, self-enroll codes	Institution-managed; limited teacher control
Edmodo	Group code + teacher approval queue	✓Best approach — but shut down in 2022

Key Insight: Most platforms rely on *security by obscurity* — a shared code is the only gate. Codes get leaked via social media, there is no identity verification, and no teacher-approval step in popular quiz tools. Edmodo’s approval-queue approach was the best but is no longer available.

Our three-part approach:

1. **Teacher approval queue** — students request to join; teachers approve/deny.
2. **Class roster matching** — students enter their roll number, matched against a pre-uploaded list.
3. **Auto-expiring join codes** — codes valid only for a limited time window.

This combination does not exist in any current lightweight quiz platform.

2. Structure of the Project with Technology Stack

2.1 Project Structure — Pages and Sitemap

```
QuizPod Platform
|
+-- Landing Page (/)
|   +-- Hero section with tagline & CTA buttons (Sign Up / Join Quiz)
|   +-- Feature cards (Practice Mode, Live Competitions, Track Progress)
|   '-- Footer with social links
|
+-- Authentication (/auth)
|   +-- Role selection toggle (Student / Teacher)
|   +-- Sign-up form (Name, Email, Password)
|   '-- Animated binary-rain background
|
+-- Student Dashboard (/student)
|   +-- Navbar (Home, XP counter, Join Quiz, Logout)
|   +-- Hero banner with stats (Rank, XP)
|   +-- Badge showcase
|   '-- Recent Quizzes grid (title, score, date, category)
|
+-- Teacher Dashboard (/teacher)
|   +-- Navbar (Home, Community, Create Quiz, Logout)
|   +-- Hero banner with stats (Total Quizzes, Upcoming Quizzes)
|   +-- Subject badges (Science, Math, English, Geography)
|   '-- Recent Quizzes grid (title, subject, submissions, date)
|
+-- Quiz Arena (/quiz/:id)
|   +-- Navbar (brand, section name, theme toggle, countdown timer)
|   +-- Question panel (text, MCQ radio options)
|   +-- Navigation (Previous, Mark for Review, Next/Submit)
|   +-- Status panel (colour-coded question grid)
|   '-- Built-in calculator tool
```

```

|
+-- Create Quiz (/teacher/create-quiz)
|   '-- Quiz builder for teachers
|
|-- Create Classroom (/teacher/create-classroom)
    '-- Classroom creation, join codes, roster, enrolment management

```

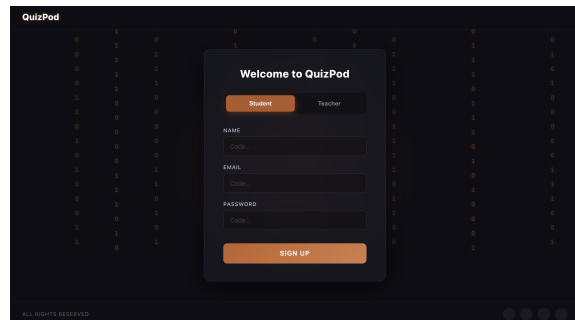


Figure 1: Frontend Screenshots — Landing Page (left) and Authentication Page (right)

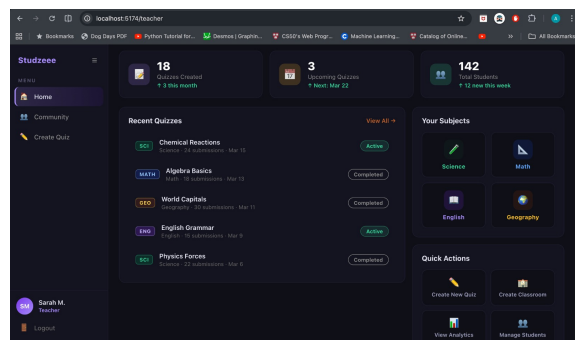
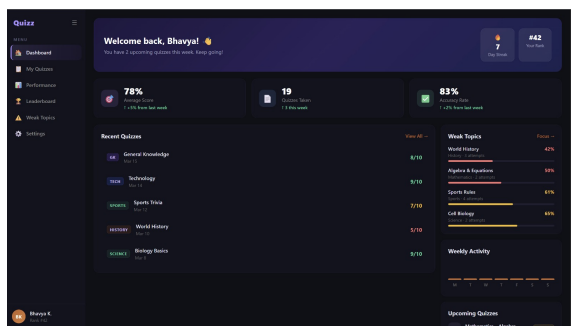


Figure 2: Frontend Screenshots — Student Dashboard (left) and Teacher Dashboard (right)

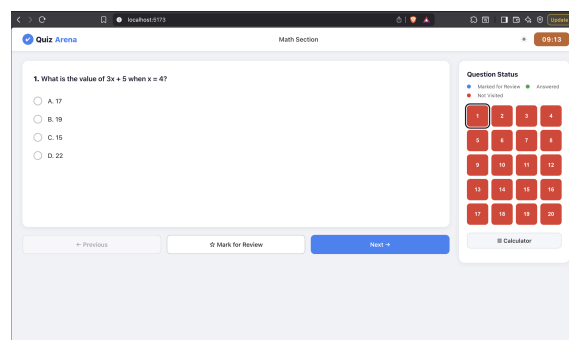


Figure 3: Frontend Screenshot — Quiz Arena

Table 3: Design Principles Applied in QuizPod

Principle	Application in QuizPod
Feedback-first, score-second	Prioritise explaining wrong answers over raw scores
No public shaming	Private leaderboards; personal stats on dashboard
Teacher time is precious	Streamlined dashboard, at-a-glance stats
Mobile-first approach	Responsive CSS for all screen sizes
Component-based architecture	Each page is a self-contained React component with its own stylesheet

Design Topology:

- **Component-based architecture** using React — modular, reusable UI components for consistency.
- **Colour-coded status system** in Quiz Arena: **green** = answered, **yellow** = marked for review, **grey** = not visited.
- **Role-based interface separation** — Student and Teacher dashboards are entirely different views with distinct layouts, stats, and actions.
- **Consistent visual language** — shared design tokens (colours, spacing, typography) across all pages.

2.2 Ideation and Brainstorming Process

1. **Problem Identification Phase:** Each team member individually used existing quiz platforms and documented pain points encountered as students.
2. **Feature Brainstorming Sessions:** The team conducted multiple brainstorming sessions over Google Meet using collaborative tools:
 - **Excalidraw** — for quick whiteboard-style sketching of page layouts, user flows, and architecture diagrams.
 - **Figma** — for higher-fidelity wireframes and component design. Mockups of Landing Page, Dashboards, and Quiz Arena were created to align the team before coding.
3. **Research Phase:** Formal research on six competing platforms to identify market gaps, with a specific deep-dive on unauthorised access prevention.
4. **Architecture Design:** Settled on a client-server architecture with clear separation between student and teacher interfaces.

Prioritised Feature Backlog:

Table 4: Feature Priority Classification

Priority	Features
High	Student & Teacher Dashboards, Create Quiz, Create Classroom, AI-generated questions, anti-cheat module, unauthorised-student detection
Low	Question tags, Mistake Book, detailed performance metrics, Coin/XP engagement mechanism

Brainstorming Artifacts: Excalidraw sketches, Figma wireframes, and Google Meet session screenshots are available in the project repository (`/assets/` folder) and can be provided on request.

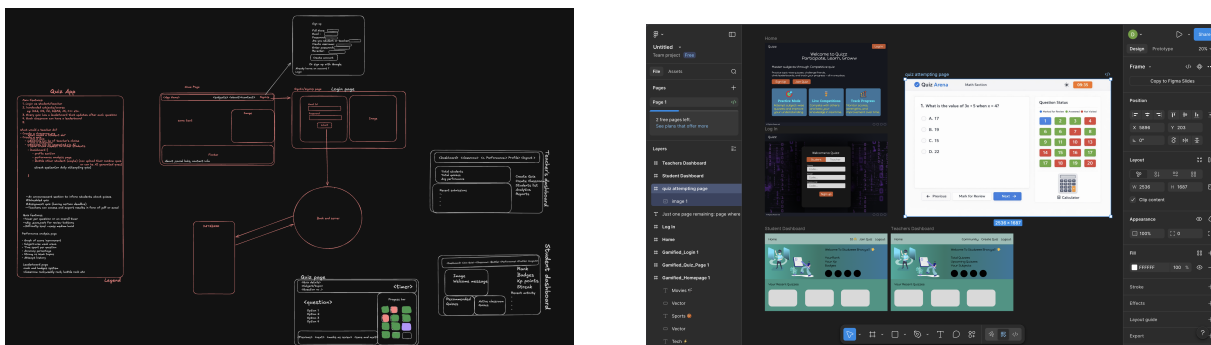


Figure 4: Brainstorming Artifacts — Excalidraw sketch (left) and Figma wireframe (right)

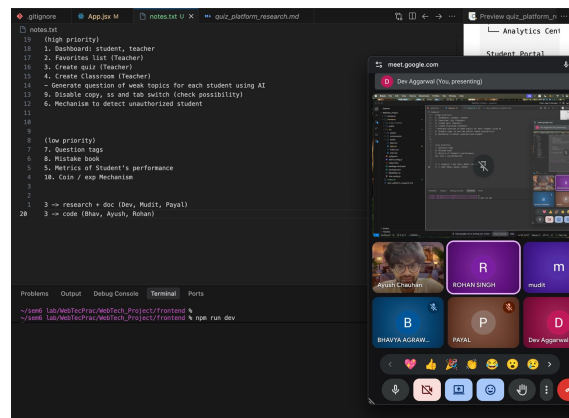


Figure 5: Team Brainstorming Session — Google Meet

2.3 Technology Stack and Justification

Table 5: Technology Stack with Justification

Layer	Technology	Version	Why This Choice?
Frontend Framework	React	19.x	Component-based architecture ideal for a multi-page app; rich ecosystem; team familiarity
Build Tool	Vite	8.x	Extremely fast dev server with HMR; near-instant rebuilds vs. Webpack
Styling	Tailwind CSS + Vanilla CSS	—	Utility-first classes for rapid styling; Vanilla CSS for complex component-specific styles
Backend Runtime	Node.js	LTS	JavaScript across the full stack; non-blocking I/O suited for real-time quiz features
Linting	ESLint	9.x	Early bug detection; enforces consistent code style across 6 team members

2.4 Deployment Plan

Table 6: Live Deployment Plan

Stage	Tool / Service	Details
Version Control	Git + GitHub	Repository at <code>DevAggarwal03/WebTech_Project</code>
Frontend Hosting	Vercel / Netlify	Free tier; automatic deployments from GitHub; global CDN
Backend Hosting	Render / Railway	Free tier; Node.js support; auto-deploy from GitHub
Database	MongoDB Atlas	Free tier (512 MB); cloud-hosted; native JavaScript driver

2.5 Data Flow Diagrams (DFD)

2.5.1 Level 0 — Context Diagram

Level 0 — Context Diagram:

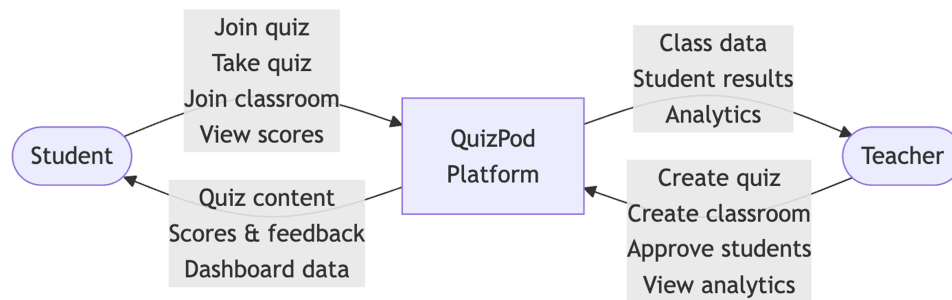


Figure 6: Level 0 DFD — Context Diagram

2.5.2 Level 1 — Major Processes

Level 1 — Major Processes:

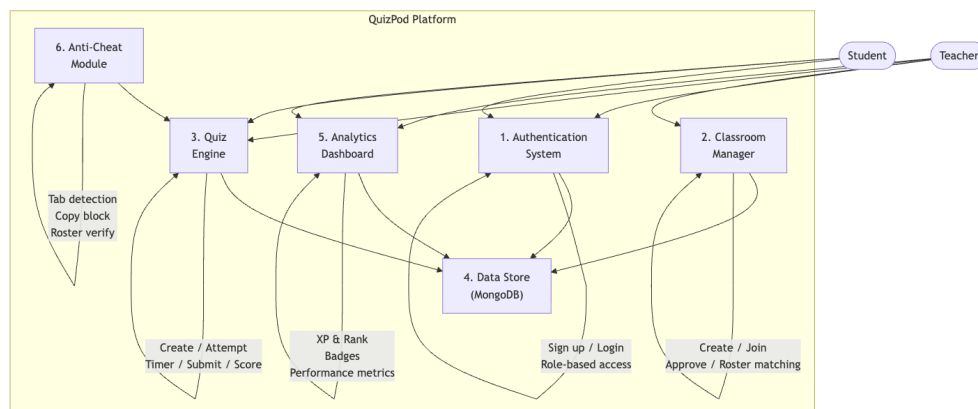


Figure 7: Level 1 DFD — Major Processes

3. Implementation

3.1 Current Implementation Status ($\approx 50\%$)

The frontend of the application has been implemented with the following completed pages and components:

Table 7: Component Implementation Status

#	Component	File	Status	Description
1	Landing Page	QuizzLanding.jsx	✓Complete	Hero section, feature cards, navbar, footer
2	Authentication	AuthPage.jsx	✓Complete	Role toggle, sign-up form, animated background
3	Student Dashboard	StudentDashboard.jsx	✓Complete	Welcome banner, XP & Rank, badges, quiz grid
4	Teacher Dashboard	TeacherDashboard.jsx	✓Complete	Stats banner, subject badges, quiz submissions
5	Quiz Arena	QuizArena.jsx	✓Complete	20 questions, timer, mark-for-review, calculator
6	Feature Card	FeatureCard.jsx	✓Complete	Reusable landing page card component
7	Backend Server	backend/index.js	✗Scaffold	Node.js initialised; implementation pending

3.2 Key Implementation Highlights

3.2.1 Quiz Arena — Full Interactive Quiz-Taking Experience

The Quiz Arena is the most feature-rich component — the dedicated area where students take their quizzes. Key features include:

- **Countdown Timer:** Real-time countdown from 9:35, with urgent styling when under 60 seconds remain.
- **Question Navigation:** Previous/Next buttons, question status grid for jumping to any question.
- **Mark for Review:** Students can flag questions to revisit, with a visual star indicator.
- **Colour-Coded Status Panel:** **Green** = Answered, **Yellow** = Marked for Review, Grey = Not Visited.
- **Built-in Calculator:** Fully functional calculator for math-related quizzes, with basic arithmetic, clear, and backspace.

3.2.2 Dual-Role Authentication

The Auth page features:

- Student/Teacher role toggle — determines which dashboard the user sees after login.
- Animated binary-rain background for visual appeal.
- Form validation with name, email, and password fields.

3.2.3 Dashboard Personalisation

Both dashboards include:

- Personalised welcome messages.

- Key stats at a glance (XP, Rank for students; Quiz count for teachers).
- Lightweight engagement: XP counter and achievement badges for students; subject badges for teachers.
- Recent quiz activity with relevant metadata.

3.2.4 Component Architecture

```
frontend/src/
+-- components/
|   +-- AuthPage.jsx           (127 lines)
|   +-- FeatureCard.jsx        (reusable card)
|   +-- QuizArena.jsx          (221 lines -- largest component)
|   +-- QuizzLanding.jsx       (88 lines)
|   +-- StudentDashboard.jsx    (106 lines)
|   '-- TeacherDashboard.jsx   (118 lines)
+-- styles/
|   +-- AuthPage.css           (4.8 KB)
|   +-- FeatureCard.css        (0.8 KB)
|   +-- LandingPage.css        (0.7 KB)
|   +-- QuizArena.css          (7.6 KB -- most detailed)
|   +-- QuizzLanding.css       (3.1 KB)
|   +-- StudentDashboard.css    (7.2 KB)
|   '-- TeacherDashboard.css   (7.8 KB)
+-- App.jsx                    (main router)
+-- main.jsx                   (React entry point)
+-- index.css                  (global styles)
'-- App.css                    (app-level overrides)
```

4. Future Work and Teamwork

4.1 Remaining Work and Next Steps

#	Task	Priority	Scope
1	Backend API — Express.js REST API with JWT auth, quiz CRUD, classroom management	Critical	Backend
2	Database integration — MongoDB schemas for Users, Quizzes, Classrooms, Scores	Critical	Backend
3	Unauthorised student detection — Roster matching, approval queue, auto-expiring codes	High	Full Stack

#	Task	Priority	Scope
4	Anti-cheat module — Tab-switch detection, copy/paste blocking, timer enforcement	High	Frontend
5	AI-powered question generation — Targeting each student's weak topics via AI APIs	High	Backend (API integration)
6	Student performance metrics — Analytics, concept-wise tracking, strengths/weaknesses	Medium	Full Stack
7	Question tags — Categorise by topic and difficulty for filtering and targeted practice	Medium	Full Stack
8	Mistake Book — Personal error log for students to review wrong answers	Medium	Full Stack
9	Coin/XP mechanism — Engagement system with XP earned from quiz participation	Low	Full Stack
10	Deployment — Vercel/Netlify (frontend), Render (backend), MongoDB Atlas (DB)	Final	DevOps

Table 8: Task Backlog for Remaining Implementation

4.2 Integration Components

Table 9: Key Integration Components

Component	Integration Details
Authentication	JWT-based auth connecting frontend AuthPage to backend API
Quiz Data Pipeline	Quiz creation → DB storage → Quiz Arena rendering → Score submission
Classroom Enrolment	Join-code generation → Student request → Teacher approval → Roster sync
AI Question Generation	Student performance data → AI API call → Personalised questions → Review centre
Real-time Features	WebSocket integration for live quiz sessions (potential)

4.3 Team Contribution

The team of 6 members is divided into two focused groups:

Table 10: Team Roles and Responsibilities

Role	Members	Responsibilities
Research + Documentation	Dev, Mudit, Payal	Platform research, competitor analysis, feature specification, documentation, UI/UX design decisions, deployment planning
Development	Bhavya, Ayush, Rohan	Frontend component development, CSS styling, backend API development, database schema design, integration

All members participated in ideation discussions, feature prioritization, and code reviews.

5. Additional Information

5.1 Key Differentiators from Existing Platforms

Table 11: Feature Comparison: QuizPod vs. Competitors

Feature	Kahoot	Quizlet	Google Forms	QuizPod
Teacher approval for enrolment	✗	✗	✗	✓
Roster-based access control	✗	✗	✗	✓
Auto-expiring join codes	✗	✗	✗	✓
In-quiz calculator	✗	✗	✗	✓
Role-based dashboards	✗	✗	✗	✓
XP & Badges	✓(speed)	✗	✗	✓(mastery)
Free tier limitations	10 players	Paywalled	Basic only	No paywall
Anti-cheat (tab detection)	✗	✗	✗	✓(planned)
AI-generated questions	✗	✗	✗	✓(planned)
Mistake Book	✗	✗	✗	✓(planned)

5.2 Project Repository

- **GitHub:** [DevAggarwal03/WebTech_Project](https://github.com/DevAggarwal03/WebTech_Project)
- **Structure:** Monorepo with `/frontend` (React + Vite) and `/backend` (Node.js)

5.3 References

1. Kahoot — <https://kahoot.com>

2. Quizlet — <https://quizlet.com>
 3. Google Forms — <https://forms.google.com>
 4. Mentimeter — <https://www.mentimeter.com>
 5. Socrative — <https://www.socrative.com>
 6. Typeform — <https://www.typeform.com>
 7. Google Classroom — <https://classroom.google.com>
 8. Moodle LMS — <https://moodle.org>
 9. Canvas LMS — <https://www.instructure.com/canvas>
 10. Edmodo (discontinued 2022) — <https://en.wikipedia.org/wiki/Edmodo>
 11. FlexiQuiz — <https://www.flexiquiz.com>
 12. ClassMarker — <https://www.classmarker.com>
 13. ProProfs Quiz Maker — <https://www.proprofs.com/quiz-school>
-